Solarflare Kernel-Bypass with CAPIO

Finn McMillan
Northwestern University
Evanston, IL, USA
finnmcm@u.northwestern.edu

Arthur Zakrzewski
Northwestern University
Evanston, IL, USA
arttomzak@u.northwestern.edu

Abstract

Kernel-bypass networking is a staple of latency-critical environments such as high-frequency trading, where per-packet context switches and the kernel's network stack provide significant cycle overhead. This increased performance comes at a cost, however: bypassing the kernel hands unprivileged processes direct pointers into NIC registers and DMA buffers, so a single memory-safety bug in the userspace driver becomes a device-control security vulnerability. We harden kernel-bypass networking with CAPIO, a capability-based I/O framework built on the CHERI architecture. Targeting a dual-port Solarflare NIC on Arm Morello, we split the driver into a thin kernel stub for hardware bring-up and a userspace driver that handles the device directly. Compiled for the purecap ABI, every pointer becomes a 128-bit capability whose bounds and permissions are enforced by hardware; the kernel grants userspace a sealed attach capability and narrowly bounded capabilities covering only the doorbell registers and DMA rings it may touch. We benchmark port-to-port round-trip latency across the two configurations: the kernel-mediated path and the userspace implementation, observing a maximum 43% decrease in Solarflare latency through CHERI kernel-bypass.

1 Introduction

In modern networking systems, two primary approaches are commonly used: traditional kernel-mediated I/O and kernel-bypass networking. Kernel-mediated I/O routes packets through the operating system kernel, offering security, isolation, and protocol support, but introducing additional latency due to system calls, context switches, and packet processing overhead within the kernel. In contrast, kernel-bypass networking maps device doorbell registers and DMA resources directly into a process' virtual address space, allowing applications to communicate more directly with the NIC and bypass the kernel's networking stack. By reducing system calls, context switches, interrupts, and packet processing overhead, kernel-bypass frameworks can achieve significantly lower latency and higher throughput metrics [5].

However, this performance gain comes at the cost of device exposure to userspace, including the mappings of packet descriptor rings, memory-mapped I/O device register pages, and pinned DMA buffers into an unprivileged, untrusted process. Here, the only protection offered to the device is the Memory Management Unit's (MMU) enforcement of memory access permissions at the 4KB granularity level. If privileged control registers happen to reside on the same page as doorbell registers, a malformed pointer can easily allow a corrupted process to take control of the NIC that it now shares an address space with [7].

The CHERI architecture, on the other hand, replaces raw integer pointers with unforgeable hardware capabilities that carry, alongside the target address, a specified set of bounds and access permissions checked on every memory reference [9]. A malformed or corrupted pointer to a doorbell register is therefore unable to compromise the device: any pointer dereference either falls within the bounds from which the capability was derived or raises a hardware exception, rather than silently capturing device memory the process was never meant to access. In effect, CHERI collapses the unit of isolation from the 4KB page down to the individual register or buffer, allowing the device interface to be exposed to userspace at exactly the granularity that the userspace requires.

We present an attack-resistant kernel bypass implementation that utilizes the CAPIO framework, on top of the CHERI Morello processor and CheriBSD operating system, to provide memory-safe userspace networking [4]. We utilize the CAPIO control plane/data plane partition, retaining management of control plane events at the kernel level while deferring data plane event work to the userspace application. We provide the userspace with capabilities to device doorbell registers and DMA resources that cannot be fabricated or expanded, allowing the application to take full control of the data plane while simultaneously being restricted from control events. We implement this application for the AMD Solarflare SFN7322F-R2, a dual-port NIC with proprietary firmware that directly supports kernel bypass [1].

Our contributions include the following:

- We identify the protection-level granularity issue as a key security vulnerability in kernel-bypass networking.
- We develop a thin kernel device stub that connects to the device through PCIe, allocates DMA resources and virtual interfaces (VIs), communicates with the NIC through its Management Controller-to-Driver Interface (MCDI), and handles interrupt-driven control events.
- We implement the CAPIO framework for use with the Solarflare SFN7322F-R2, adapting the device manifest and slicer mechanisms for compatibility with the Solarflare.
- We develop a user-level driver for the SFN7322F-R2 that handles transmission and reception of Ethernet packets that abides by the CAPIO framework.
- We create a lightweight testing harness for benchmarking of Solarflare CAPIO kernel-bypass networking and memory security, comparing the latency of kernel and userspace datapaths alongside capability safety testing.
- We identify key overlaps between the CAPIO framework and existing Solarflare virtualization functionality.

2 Background

We cover CHERI's capability-based memory model and the ARM Morello platform. This is followed by an overview of existing kernel-bypass networking architectures as well as a deeper dive into the Solarflare hardware architecture we work with.

2.1 CHERI

CHERI (Capability Hardware Enhanced RISC Instructions) extends 64-bit pointers in conventional architectures to 128-bit, hardware-enforced capabilities, which grant specific access rights to bounded memory regions [9]. A capability holds a 64-bit address alongside a compressed description of the lower and upper bounds it authorizes, an architectural permissions mask, and an object-type field. Each capability additionally carries a single validity tag bit, held out of band in tagged memory and registers rather than within the 128-bit value itself. This tag is preserved only by capability-aware loads and stores; any ordinary, non-capability write to the same location automatically clears it. Because integer data can never set the tag, software cannot fabricate a capability from arbitrary bytes, making capabilities unforgeable. Additionally, capabilities are also constrained to be monotonic: a process that was handed a tightly bounded capability may subdivide it for internal use, yet cannot widen its range or increase its permission set. Every load, store, and instruction fetch through a capability is enforced by the processor: any attempt to access memory outside of the capability's bounds, perform an unpermitted operation, or dereference an untagged value results in a hardware exception. This results in a drastic increase of spatial memory safety, subsequently preventing common memory-based vulnerabilities such as buffer overflows, out-of-bounds accesses, and pointer arithmetic errors.

CHERI's sealed capabilities extend this model to encapsulation. Sealing binds a traditional capability with a specific object type, preventing the capability from being dereferenced or modified [10]. Furthermore, an application is only able to unseal such a token when in possession of the corresponding object type capability. This methodology allows sealed capabilities to act as opaque authentication tokens, where applications must possess proper authentication from a trusted source to pass verification. Sealed capabilities are crucial to the CAPIO framework, allowing a secure connection to be established between a userspace process and the kernel.

2.2 The ARM Morello Platform and CheriBSD

These CHERI primitives reach software through the pure-capability (purecap) ABI, under which every pointer in a program, including code pointers and stack references, is a capability, such that all memory accesses must be verified by the processor. CHERI's most prominent silicon implementation is known as Morello, Arm's prototype CHERI-extended Armv8.2-A processor. The most mature CHERI operating system, CheriBSD, is an adaptation of FreeBSD, providing a purecap kernel and userspace [10]. CAPIO in particular is built on CheriBSD, and makes important modifications to the mmap system call to support the slicing framework.

2.3 Kernel-Bypass Architectures

To eliminate system call overhead and the kernel's heavy networking stack, applications operating in high-performance environments utilize frameworks such as DPDK and ef_vi. DPDK, or Data Plane Development Kit, is an open-source Linux Foundation software for kernel-bypass networking, used as the basis for native NIC Poll Mode Drivers (PMDs) such as NVIDIA's Mellanox mlx5 [6]. Ef_vi, on the other hand, is AMD Solarflare's proprietary kernel-bypass API, operating at the Data Link layer of the OSI networking model [2].

DPDK and ef_vi differ in detail but share an operational model common to most kernel-bypass frameworks. The NIC exposes a set of descriptor rings, or circular queues of buffer descriptors, in memory shared between the device and the application. To receive, the application posts physical addresses to empty buffers onto a receive ring, and to transmit, the application constructs a packet in a DMA buffer and posts a physical address to it on its transmit ring. It notifies the device by writing to a doorbell register mapped into its address space, drastically decreasing latency by avoiding a system call. The userspace driver also operates in the absence of interrupts within the data path: in DPDK, the device writes status into descriptor objects themselves, whereas in Solarflare and Mellanox hardware, events are written into a completion queue that is polled by the application. Additionally, packet buffers are DMA targets, and therefore must be pinned in physical memory.

This work attempts to recreate a CHERI-conscious version of ef_vi, the lowest-level userspace interface for Solarflare networking cards. Ef_vi utilizes a central abstraction known as the virtual interface, which packages a select window of dataplane resources such as receive and transmit rings, an event queue, and a page of doorbell registers. This bundle can then be mapped into a userspace application's address space, simultaneously avoiding exposure of privileged device registers. A single adaptor can also support the creation of many VIs; hardware filters, which are installed through commands issued by the control plane kernel stub, take responsibility for steering packet traffic to its proper VI.

While this protection model offers a level of security required to make kernel bypass feasible, it is not without faults. The VI abstraction protects control plane resources from the application, but still operates at page-level granularity, requiring data and control registers to be placed on different pages in memory. Within the userspace process, spatial memory errors are still real and commonplace: the application holds raw pointers to device resources, causing a stray write or buffer overflow to corrupt data on the wire without any fault within the program. While ef_vi is able to achieve many of the same security benefits as CAPIO through the use of VIs, it requires extensive firmware utilization from the Solarflare NIC to support it.

2.4 The Solarflare EF10 Family

The EF10 is the controller architecture underpinning Solarflare's SFN7000- and SFN8000-series adapters, which span the Huntington and Medford silicon [1]. Its datapath is organized around the virtual interface, where each VI is assigned an 8 KB window within the device's memory-mapped register region. The control plane, on the other hand, is cleanly separated from data: all privileged configuration, such as allocating queues, inserting filters, and setting MAC and link parameters, is issued through the MCDI. The MCDI acts as a request/response channel between the host driver and the firmware on the device's management controller, responsible for all hardware bring-up and configuration activity.

The EF10 family also supports crucial functionality known as SR-IOV, or Single Root I/O Virtualization, which allows for partitioning

of a single device among multiple independent users. Each port on the NIC is mapped to a physical function (PF) that can then spawn multiple virtual functions (VFs). While a PF acts as a fully-equipped PCIe endpoint that can issue privileged operations, a VF acts as a lighter weight, less privileged endpoint that carries its own PCIe requester ID and memory-mapped resources. Each VF carries its own set of VIs and doorbell windows, and appears to the system as a unique NIC that can be passed to a distinct virtual machine or process. While the VI abstraction offers process or thread-level isolation on doorbell writes, VFs offer DMA-level isolation, which is crucial for multi-tenant systems.

3 The CAPIO Architecture

The CAPIO framework, designed on top of the CHERI architecture, was originally created to enable kernel bypass on traditional NICs that lacked special firmware functionality. Oftentimes, these devices, such as the Intel E1000e, map control plane and data plane registers to shared pages in memory [8]. When exposed to userspace, this poses a significant security risk, as a corrupted userspace process with access to the device's data plane can simultaneously take full control of the NIC. The CAPIO architecture addresses this issue by targeting the protection granularity problem using CHERI capabilities, alongside five other primary components: a pure capability kernel, the kernel stub, the Interface, the userspace driver, and the Slicer [4].

3.1 The CAPIO Kernel Framework

CAPIO requires both a pure capability operating system kernel, as well as kernel-level driver stub for the target device. This pure capability kernel, like the one supported by the CheriBSD operating system, acts as the root of trust for the system and governs access to all devices [3].

Alongside the kernel itself is a thin, device-associated kernel stub, whose attachment to the kernel marks the beginning of the CAPIO lifecycle. This stub takes responsibility for control plane concepts such as hardware initialization, device memory allocation, and interrupt-driven control events. Alongside standard responsibilities, the kernel stub also defines a CAPIO Device Manifest, detailing the device's register layout and specific access permissions. Upon completion of shared memory region definition and firmware initialization, the kernel stub then registers these device resources with the Interface, which provides an API to control userspace access to devices.

3.2 The CAPIO Userspace Implementation

The device's primary driver is a userspace program that attaches to the device through CAPIO. This application is an unprivileged, capability-aware process that controls the data plane hot path. To begin, the userspace driver calls *CAPIO_ATTACH*, as seen in Figure 1. If this call succeeds, it receives a sealed capability from the kernel that verifies its role as the device's driver. The program subsequently uses this token to request mappings to device memory through the Slicer. Rather than allowing the application to issue standard mmap system calls for the device, the Slicer intercepts mmap requests and returns bounded capabilities to device resources, dictated by the kernel's Device Manifest. Due to the unforgeable nature of these capabilities, the userspace driver is strictly prevented from accessing any control state registers that lie outside of its issued capability slices.

```
sfc->cap_token = malloc(PAGE_SIZE); // empty page to verify
↪  ownership of memory

sfc->cap_req.user_cap = sfc->cap_token;
if (ioctl(sfc->fd, CAPIO_ATTACH, &sfc->cap_req) < 0) {
    goto fail;
};

sfc->tx_buffer = map_region(sfc, SFC7120_TX_BUFFER, NULL,
↪  NULL);
if (sfc->tx_buffer == NULL) {
    goto fail;
}

sfc->rx_buffer = map_region(sfc, SFC7120_RX_BUFFER, NULL,
↪  NULL);
if (sfc->tx_buffer == NULL) {
    goto fail;
}
```

Figure 1: Example of mapping device memory through a capability token verified through the CAPIO_ATTACH ioctl in kernel space

Once this connection has been established, the userspace driver is able to control transmission and reception of packets in the hot path through its assigned capabilities without kernel intervention. If the application requires any sort of privileged action to be taken regarding the device, such as configuration of DMA registers, it must make a request to the kernel stub, passing its sealed capability as authentication for the action.

4 Implementation

We implemented kernel-bypass user-level networking architecture for the Solarflare SFN7322F-R2 built on top of the existing CAPIO framework originally implemented for the Intel e1000e. Our implementation consisted of a thin kernel driver stub, a user-level networking driver, and small changes to the existing CAPIO framework. The substantially more complex hardware architecture behind the Solarflare NIC served as a stress test for the extensibility of the CAPIO architecture, and a proof of concept for the future of high-performance networking.

4.1 The Kernel Stub

The purpose of the thin kernel stub is to handle all of the privileged work living within the control plane that lives outside of the hot path of packets in a slim form factor. The stub handles the PCI lifecycle, brings up the device through the MCDI firmware interface, sets up the control event interrupt path, and initializes shared memory regions for DMA resources of one virtual interface, which are eventually exposed to the userspace.

In contrast to the e1000e NIC used in CAPIO, the hardware interface is far more complex and is fundamentally different in the

way data travels through it. The Solarflare SFN7322F-R2 is an event-driven NIC which utilizes virtual interfaces (VIs) to provide a private channel between a process and NIC resources, and uses descriptor rings, an event queue, and doorbell registers to communicate. One VI consists of an event queue, transmit and receive descriptor rings, and a doorbell window, paired alongside a packet buffer. The kernel stub allocates these resources as DMA-coherent memory alongside the MMIO doorbell register, which eventually get handed up into the userspace following the minting of bounded CHERI capabilities, with the exception of an interrupt-driven event queue remaining in the kernel-space. Additionally, information surrounding buffer addresses, ring sizes, head pointers, and more are exposed through an ioctl to give the userspace everything it needs to communicate with the NIC.

This two-event-queue design allows for purely data events to populate the pollable userspace event queue for the sake of security and performance, while rarer control-plane events communicate through interrupts. This allows us to avoid expensive system calls and context switches on the per-packet hot path, while keeping the userspace networking streamlined and free from having to decode control-plane events.

4.2 Userspace Migration

Although the end goal was complete kernel bypass, we first implemented the packet path entirely within the kernel through transmit and receive ioctl calls. Buffer copies, descriptor writes, doorbell rings, and event-queue polling were verified in a test harness on our hardware before the addition of the complexities that came with true userspace networking. Additionally, this opened the floor for a concrete frame of reference when analyzing the difference in performance following the hot path being moved to the userspace with the use of capabilities.

4.3 The Userspace Driver

We developed a userspace driver mimicking the essential functionality of ef_vi, AMD's Solarflare networking API, which maps the VI resources into the process and allows for round trip user-level networking backed by the security of bounded capabilities. At initialization, the userspace driver attaches to the device, maps the resource memory regions into the process accessed through CHERI capabilities (as previously seen in Figure 1), and reads the VI info through an ioctl call.

Transmitting a packet sees *sfc7120_tx_post*, as shown in Figure 2, copy its frame into the packet buffer slot, build the transmit descriptor using our kernel-supplied address, and promptly ring the NIC's doorbell register through the MMIO region, and return. Following the capability-enabled doorbell ring, the NIC knows to send out the packet and populate the event queue with a confirmation event.

Receiving a packet utilizes *poll* to remain in a tight loop waiting to decode a receive event within the user-level data event queue. Following this detection, *sfc7120_rx_recv* reads the filled slot in the packet buffer dictated by the receive head pointer, re-posts the descriptor, and again rings the NIC's doorbell to return the buffer to the NIC for use.

In this model, each of the three key functions, poll, rx_recv, and tx_post, own the event queue, receive descriptor ring, and transmit descriptor ring respectively, creating a frictionless data path entirely in userspace. All while bounded capabilities prevent silent invalid memory accesses from occurring at any point of the hot path.

```
slot = sfc->tx_buffer + sfc->tx_head *
↪  SFC7120_TX_BUFFER_SIZE;
memcpy(slot, buf, len); // copy frame into buffer

// build the 8-byte descriptor
pa = sfc->vi_info.tx_buffer_paddr + sfc->tx_head *
↪  SFC7120_TX_BUFFER_SIZE;
tx_ring[sfc->tx_head] =
    ((uint64_t)(len & 0x3fff) << 48) | // byte count
    ((uint64_t)((pa >> 32) & 0xffff) << 32) | // high 16
    ((uint64_t)( pa & 0xffffffff)); // low 32

wptr = (sfc->tx_head + 1) & (SFC7120_NUM_TX_DESC - 1); //
↪  new write pointer
__asm__ volatile("dsb sy" ::: "memory"); // memory barrier
↪  for ordering
((volatile uint32_t * __capability)
    sfc->mmio_slices[SFC7120_SLICE_TX_DESC_DBL].addr)[2] =
    ↪  wptr; // ring doorbell through CHERI capability and
    ↪  alert NIC

sfc->tx_head = wptr; // update local state
```

Figure 2: Key functionality of the sfc7120_tx_post function within our userspace driver.

4.4 Updates to CAPIO Framework

Bringing the NIC resources to userspace with capabilities surfaced a bug in the existing CAPIO framework's attribute assignment of regions backed by device registers. In its previous implementation for the e1000e NIC, CAPIO mapped these regions with the *VM_MEMATTR_UNCACHEABLE* attribute which defined it as regular uncacheable memory, as opposed to the stricter, more strongly ordered *VM_MEMATTR_DEVICE* attribute. Reads of canary registers that worked when the kernel had ownership of the resources were silently returning a 0 value without an expected exception, until the stricter memory attribute was enforced. The work on the simpler e1000e NIC tolerated the weaker attribute, while the Solarflare architecture did not.

4.5 Observations

Through the extension of the CAPIO architecture to new hardware we observe that despite the Solarflare hardware architecture being considerably more complex than that of the Intel e1000e, this was a relatively thin task. As Table 1 shows, the existing CAPIO framework was nearly unchanged, and the additional 1521 lines of code over the e1000e CAPIO stub [4] can be attributed to the increased hardware complexity seen in the Solarflare NIC, specifically the MCDI firmware layer.

Additionally, we observe that in some ways, Solarflare's hardware-powered VIs and CAPIO solve the same problem of preventing a process from accessing restricted memory. However, this guarantee provided through CAPIO serves as a proof-of-concept that is not

Table 1: Implementation Effort by Component (Source Lines of Code)

Component	SLOC
Kernel Stub	2,721
Userspace Driver	405
Changes to CAPIO Framework	62
Total	**3,188**

rooted in the silicon design of the NIC, but rather in the host's capability hardware which provides more granular, byte-level security. This suggests that assuming there doesn't exist a notable performance tradeoff, the future of high-performance NICs could move away from a reliance on proprietary hardware-based memory access security solutions, and rather rely on host hardware.

5 Evaluation

We now evaluate the effectiveness of our CHERI CAPIO userspace driver for the Solarflare SFN7322F-R2. In particular, our implementation aims to answer the following questions:

- Does the userspace networking path for Solarflare EF10s provide significant benefits in latency as opposed to the traditional kernel-mediated path?
- Does the CAPIO framework and enforcement of bounded capabilities on device resources successfully enforce device and program security?
- How do 128-bit CHERI capabilities impact the latency metrics of Solarflare kernel bypass, as opposed to traditional architectures?

Due to time constraints, the final research question regarding the impact of CHERI capabilities on latency was unable to be tested in a formal environment, and must be delegated to future work.

5.1 Experimental Setup

Our evaluation was performed on an ARM Morello Development Board running four CHERI-enabled Neoverse N1 ARMv8.2-A processors. We installed a single Solarflare SFN7322F-R2 dual-port 10 GbE adapter running the low-latency datapath firmware variant [1]. This dual-port card was exposed as two PCIe physical functions, designated as PF0 and PF1, connected port-port by an SFP+ direct-attach cable. Every frame measured in this setup therefore crosses a real 10 GbE link.

Our test harness was designed as a single-process application that attaches to both physical functions and bounces raw Ethernet frames across the wire and back: it transmits on PF0, reflects the frame at PF1, and timestamps the packet's arrival back at PF0. Due to both endpoints living in a shared address space, the round-trip time (RTT) is measured using a single monotonic clock.

In our latency test, 1,000 warm-up round trips were first conducted to eliminate cold-start speed penalties, and 20,000 RTT samples were collected. Minimum, mean, and p99 metrics were aggregated across these samples for frame sizes of 64, 1028, and 1518 bytes.

As expected, the userspace data path experienced significant latency benefits over traditional kernel-mediated networking. The kernel-bypass implementation experienced the most substantial speedup on smaller frame sizes, achieving a 43% gain over the kernel on frame sizes of 64 bytes as opposed to only a 27% speedup on frames of 1518 bytes.

One of the strongest metrics observed in this experiment setup was consistency: in both the kernel and userspace paths, p99 values sat roughly within 5% of minimum round trip times. This can be attributed to the synchronous, busy-polling setup of the experiment, which removes variation in speeds introduced by the scheduler or interrupt handler. The security of the CAPIO framework was also tested through a sequence of simple memory attacks on DMA buffers and device doorbell registers.

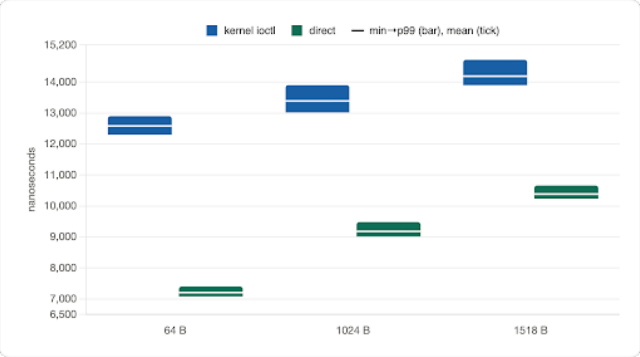

Figure 3: Latency comparison between userspace and kernel data path.

5.2 Security Testing

To validate the security claims of CHERI and CAPIO in the userspace, we also constructed a security test suite that commits memory-safety violations against each class of capability that the kernel grants through resource mapping. Three violations were tested, covering the three resource types that a CAPIO userspace driver holds: an out-of-bounds store on a sliced MMIO doorbell capability, a one-byte buffer overflow past the end of a 1 MB-bounded transmit packet-buffer capability, and an off-by-one indexing error that writes one descriptor past the 512-entry transmit descriptor-ring capability. Each violation executes inside of a forked child process, so that the expected fault terminates only the child. The parent process then inspects the child's exit status and reports whether the access was trapped or silently permitted.

In all three cases, the violation raised *SIGPROT*, CheriBSD's capability-protection signal, before the access could corrupt device state or DMA memory [3]. These tests successfully demonstrate the use of CAPIO as a spatial-memory safety mechanism, confirming that a compromised userspace driver cannot reach memory-mapped I/O registers or DMA buffers beyond those that were explicitly delegated to it.

Importantly, these tests identified that CHERI successfully enforces memory safety at the CPU level. It is crucial to note that

capability protections do not extend to DMA, and thus a malicious process could still corrupt NIC state by writing a bad physical address to a descriptor if the Solarflare was operating in physical mode [1].

6 Future Work

We cover multiple promising directions for extending our implementation, spanning additional performance benchmarking, hardware feature adoption, and performance engineering potential.

6.1 Benchmarking on AArch64

As noted in our Evaluation section, one crucial area of investigation that was unable to be realized within our time constraint is benchmarking of our Solarflare userspace driver against standard AArch64. As previously stated, one of the primary research objectives of this project was to compare the security improvements offered by CHERI and CAPIO with potential latency overhead introduced by the use of capabilities. Future work on this project should focus on the development of a non-CAPIO, standard AArch64 executable userspace driver; one that can be benchmarked using the same test harness as described in this paper. Capabilities, as opposed to standard pointers, require enforcement of bounds and permission levels by the hardware on every memory access, which could result in additional cycles needed on the hot data path. Additionally, capabilities occupy twice as much space as a regular 64-bit pointer, resulting in increased pressure on both the cache and the TLB. For these reasons, benchmarking standard AArch64 Solarflare userspace networking against AArch64c is an important step in reasoning about the feasibility of CHERI and CAPIO in large-scale kernel bypass.

6.2 Solarflare's Buffer Mode

By design, the Solarflare EF10 family features two different methods of translating packet buffer addresses: *Physical Addressing Mode* and *Buffer Table Mode* [1]. Our current implementation utilizes Physical Addressing Mode, where the NIC expects raw physical addresses pointing to packet buffers to which it issues DMA commands. While optimal for latency and large-sized buffer pools, this implementation offers no memory protection; the userspace is able to direct the device towards any physical address it chooses, regardless of what data resides there. CHERI Capabilities are also of no use here, as their scope does not extend beyond CPU loads and stores. Future work should focus on redesigning the driver to work in Buffer Table Mode, which utilizes an on-chip hardware buffer table containing physical addresses to DMA regions. This buffer table is populated through the trusted kernel stub, and forces the userspace to specify an index into the buffer table within each descriptor it writes, rather than a raw address. Enforcing this mode on the SFN7322F-R2 aligns with this project's goal of memory-safe kernel bypass, and should thus be implemented as a next step.

6.3 Concurrency With Multiple VIs

The current implementation drives the resources of a single Virtual Interface, and utilizes only one event queue, one transmit descriptor ring, and one receive descriptor ring. While this kept our initial implementation simple, each physical port on this Solarflare NIC supports 1024 Virtual Interfaces [11] which invites the possibility of concurrently driving multiple VIs and distributing traffic across them as necessary, increasing packet throughput potential and strengthening the argument for a migration to capability-based security. The byte-granular capability model would make this design extension clean, and could allow for the registers of many distinct VIs to be densely packed onto a single page, decreasing the MMIO footprint while improving packet throughput.

6.4 Optimizing Packet Buffer

Although our implementation removes the kernel from the data path, packet memory copies are still performed on both the post and receive sides due to our current implementation of the packet buffer. Currently *sfc7120_tx_post* copies the frame to be sent into a DMA buffer slot, and *sfc7120_rx_recv* copies the received packet out of the slot after its population in the DMA buffer. Alternative kernel-bypass networking architectures like DPDK utilize copy-free memory pools, a model that naturally fits the CHERI capability model and is a logical next step for performance engineering our architecture.

7 Conclusion

We expanded on the existing CAPIO architecture to support the Solarflare SFN7322F-R2, a significantly more complex NIC than the Intel e1000e it was initially implemented for. While the e1000e exposes data and control planes through register pages, the Solarflare communicates through an event queue, descriptor rings, and doorbell registers. This architecture mapped cleanly to the control-plane and data-plane split present in CAPIO, with a thin kernel stub primarily handling privileged control-plane activity, while the userspace driver entirely handles the packet hot path, accessing NIC resources through bounded CHERI capabilities. Adapting existing CAPIO infrastructure required minor changes due to the more advanced NIC hardware's increased strictness on device memory ordering. This proves the extensibility of the CAPIO framework to high-performance NICs designed around kernel-bypass networking.

We evaluated our architecture in terms of the security CHERI capabilities provided and the performance gains that came with moving the data path to the userspace. We saw our capabilities successfully trap every out-of-bounds access to a doorbell, packet buffer, or descriptor ring with a SIGPROT. Additionally, the userspace data path saw a 43% decrease in round-trip latency on 64 byte frames and a 27% decrease in round-trip latency on 1518 byte frames when compared to our ioctl-based kernel-space path. Future work leaves room for utilizing Solarflare's buffer mode, performance benchmarking without the use of capabilities, and further performance engineering efforts through the concurrent use of multiple VIs and a transition to a copy-free data path to enhance the security claims and solidify the performance argument for our capability-based architecture.

Although CHERI capabilities within Solarflare CAPIO introduce some degree of security redundancy, this implementation serves as a powerful proof of concept. The idea that both host hardware and NIC silicon features can deliver similar protection against unauthorized memory accesses, provided future benchmarking confirms

the lack of a significant performance penalty, could prove the viability of host-based memory security within high-performance networking.

8 Acknowledgements

The authors thank the following people for their contributions to this work:

- Friedrich Doku for his frequent support throughout the project, assistance with onboarding onto Prescience Lab hardware, and valuable insights from his prior CHERI work.
- Josh Brice for providing us with the Solarflare NIC, and providing guidance and direction during the early stages of the project.
- Prof. Peter Dinda for his encouragement and support throughout the quarter.

References

[1] Inc. Advanced Micro Devices. [n. d.]. *Solarflare Server Adapter User Guide.* AMD. https://www.amd.com/content/dam/amd/en/support/downloads/solarflare/drivers-software/SF-103837-CD-28_Solarflare_Server_Adapter_User_Guide.pdf

[2] Advanced Micro Devices, Inc. 2026. Onload User Guide (UG1586). https://docs.amd.com/r/en-US/ug1586-onload-user/ef_vi. Document ID: UG1586, Revision 1.5.

[3] Brooks Davis. 2016. CheriBSD: A Research Fork of FreeBSD. In *Proceedings of AsiaBSDCon 2016.* Tokyo, Japan. https://papers.freebsd.org/2016/asiabsdcon/brooks-cheri/

[4] Friedrich Doku, Jonathan Laughton, Nick Wanninger, and Peter Dinda. 2025. CAPIO: Safe Kernel-Bypass of Commodity Devices using Capabilities. arXiv:2512.16957 [cs.CR] https://arxiv.org/abs/2512.16957

[5] Matthias Jasny, Muhammad El-Hindi, Tobias Ziegler, and Carsten Binnig. 2025. A Wake-Up Call for Kernel-Bypass on Modern Hardware. In *Proceedings of the 21st International Workshop on Data Management on New Hardware (DaMoN '25).* Association for Computing Machinery, New York, NY, USA, Article 14, 5 pages. doi:10.1145/3736227.3736235

[6] The DPDK Project. [n. d.]. About DPDK. Retrieved June 9, 2026 from https://www.dpdk.org/about/

[7] Konstantin Taranov, Benjamin Rothenberger, Daniele De Sensi, Adrian Perrig, and Torsten Hoefler. 2022. NeVerMore: Exploiting RDMA Mistakes in NVMe-oF Storage Applications. arXiv:2202.08080 [cs.CR] https://arxiv.org/abs/2202.08080

[8] The Linux Kernel Development Community. [n. d.]. Linux Base Driver for Intel(R) Ethernet Network Connection. https://docs.kernel.org/networking/device_drivers/ethernet/intel/e1000.html. Linux Kernel Documentation, v7.1.0-rc7.

[9] Robert Watson, Jonathan Woodruff, Peter Neumann, Simon Moore, Jonathan Anderson, David Chisnall, Nirav Dave, Brooks Davis, Khilan Gudka, Ben Laurie, Steven Murdoch, Robert Norton, Michael Roe, Stacey Son, and Munraj Vadera. 2015. CHERI: A Hybrid Capability-System Architecture for Scalable Software Compartmentalization. 2015 (05 2015). doi:10.1109/SP.2015.9

[10] Robert N. M. Watson, Simon W. Moore, Peter Sewell, and Peter G. Neumann. 2019. *An Introduction to CHERI.* Technical Report. University of Cambridge Computer Laboratory. https://www.cl.cam.ac.uk/techreports/UCAM-CL-TR-941.pdf

[11] Inc. Xilinx. [n. d.]. *Solarflare Server Adapter User Guide.* Xilinx. https://www.amd.com/content/dam/amd/en/support/downloads/solarflare/onload/openonload/packages/SF-114063-CD-10_ef_vi_User_Guide.pdf

A AI Usage

Our team heavily utilized AI throughout our project, leaning heavily on tools such as Claude Code and Gemini's Deep Research tool. To begin our understanding of the project and key dependencies, we used Gemini to conduct a deep dive into subjects such as CAPIO, kernel bypass, CHERI, and Solarflare networking, supplying direct resources such as publications or documentation when necessary. Using this particular tool allowed us to gain the level of expertise necessary to drive our subsequent coding agents with authority and understanding.

The team utilized Claude Code for practically all of the manual programming work of this project, opting to focus on design decisions and tradeoffs as opposed to raw coding output. Claude Code's internal tools, such as skills and CLAUDE.md files, were heavily utilized to bring structure and encapsulation to such a large code base.

Overall, Claude Code was an especially helpful tool; without it, this project simply would not have been able to cover the scope that it does in the restricted time frame given. Had it not been for Claude Code, we would have been forced to spend large amounts of time dealing with dependencies, linking, build configurations, and more, hindering our ability to focus on the crucial concepts in memory safety and networking that the project actually targets.

B Finn's Thoughts on AI Use

Overall, in terms of my AI usage, I'd attribute 80% to work done inside of Claude Code and 20% utilizing Gemini's deep research tool for in-depth overview of key project concepts. This project served as my first introduction to low-level development: my previous experience with device drivers and kernels was purely theoretical and incredibly high-level. Without the use of AI tools, attempting a project of this scale and domain would be unfeasible. Overall, I believe that the Claude Team account purchased for the use of this class was an incredibly good use of funds, as it allowed us to gain hands-on experience building a project in a domain that we would otherwise have never been introduced to.

C Arthur's Thoughts on AI Use

I spent a vast majority of my time working on this project with Claude Code, paired with a considerable amount of time with the Google Gemini chat and deep research functions in the earlier stages when we were ironing out the ultimate direction of the project, and analyzing the potential redundancy between VIs and capabilities. According to my Claude Code insights, I logged 68 hours of coding with Claude Code and personally felt as if it was incredibly helpful in terms of debugging, writing repetitive code, and quickly understanding existing CAPIO and on-tree Solarflare driver code. I feel as though it definitely let me learn more than otherwise would have been possible given my level of experience with low-level development, and gave me the confidence to tackle a project of this scope. I believe that the paid Claude Team account was a great use of funds.